Name: Nurul Iman Batrisyia Binti Mohd Romizan

Student ID: 25014830

Assignment 2

1. Why does BFS ignore the distances stored in the graph even though they are available to it?

   - Breadth-First Search (BFS) treats the graph as unweighted, focusing purely on finding the shortest path in terms of the **fewest number of edges** (hops) rather than edge weight/cost. It explores all neighbor nodes at the current depth level before moving deeper, making edge costs irrelevant to its queue expansion logic.

2. Which two lines change when bfs is turned into dfs, and which of them makes DFS order dependent?

   - Changing BFS (which uses a FIFO Queue) into DFS (which uses a LIFO Stack) typically involves changing the data structure retrieval/insertion method from a queue to a stack (or changing pop/append operations).The line handling the order of node expansion or child insertion makes DFS order-dependent, because depending on whether you push left-to-right or right-to-left onto a stack, the last-inserted child gets popped first, drastically altering the traversal path.

3. UCS tests for the goal when a state is popped from the heap. What could go wrong if it tested when the state was first generated?

   - If Uniform Cost Search (UCS) tested for the goal upon generation rather than upon popping, it could return a suboptimal (more expensive) path. A state might be generated early via a high-cost path, but a cheaper, optimal path to that same state might be discovered later before it is popped from the priority queue. Testing at expansion ensures the lowest cumulative cost $g(n)$ to that state has officially been reached.

4. Lab1reports *expanded* and *runtime_ms*. Which one should you use to compare BFS and UCS with a classmate's run on a different laptop, and why?

   - You should use expanded (number of nodes expanded). runtime_ms depends heavily on hardware specifications, CPU clock speed, current background processes, and implementation differences across different

laptops, making it an unfair metric. expanded measures algorithmic efficiency and node evaluations independently of hardware performance.

5. Which quantity does GBFS ignore that A⋆ uses, and which failure in Lab2 does that explain?
   - Greedy Best-First Search (GBFS) ignores the path cost-so-far $g(n)$, relying solely on the heuristic $h(n)$. This explains why GBFS can fail to find the optimal path or get stuck in dead ends/loops, because it blindly chases the apparent closeness to the goal without accounting for the actual cost incurred to get there.

6. Frontier states *X* and *Y* have *(g, h) = (8,2)* and *(3,6)*. Which does UCS expand first? GBFS? A⋆?

   - State X: $g = 8, h = 2 \Longrightarrow f(X) = g + h = 10$
   - State Y: $g = 3, h = 6 \Longrightarrow f(Y) = g + h = 9$
   - UCS (Lowest $g$ first): Expands State Y first ($3 < 8$).
   - GBFS (Lowest $h$ first): Expands State X first ($2 < 6$).
   - A⋆ (Lowest $f = g + h$ first): Expands State Y first ($9 < 10$).

7. Why must *h* be expressed in the same units as the edge costs?

   - In algorithms like A⋆, the evaluation function is calculated as $f(n) = g(n) + h(n)$. Since you are adding path cost ($g$) and heuristic estimate ($h$) directly together, they must share the exact same dimensional units (e.g., kilometers, minutes, or currency) to make mathematical sense and maintain admissibility/consistency.

8. Lab2 computes $h\star$ by running Dijkstra on the reversed graph from the goal. Why reversed?

   - Running Dijkstra from the goal node on a **reversed graph** (where all edge directions are flipped) efficiently computes the exact shortest path distances from *every* node in the graph *to* the goal in a single search pass. This gives you the exact optimal heuristic values ($h^\star$) for all states simultaneously.

9. Weighted A⋆ with *w = 1.5* changed the cost for KUL→KEF but not for SIN→EZE. What property of the SIN→EZE route explains this?

   - The property of the SIN→EZE route that explains this is that the standard A⋆ path found for SIN→EZE was already the absolute optimal (shortest possible) path, meaning even when weighted down by a suboptimal

multiplier ($w = 1.5$), no cheaper or alternative sub-optimal shortcut existed that Weighted A$\star$ could greedily prefer over the true optimum.